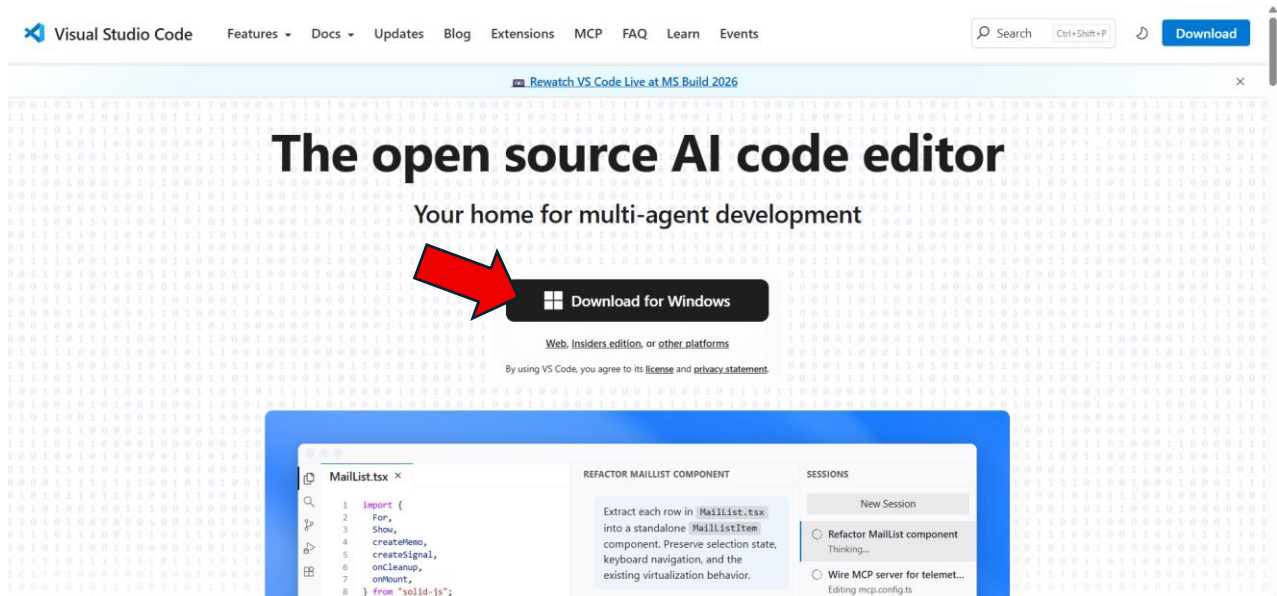


User Manual (Return Risk Prediction)

(คู่มือการใช้งาน)

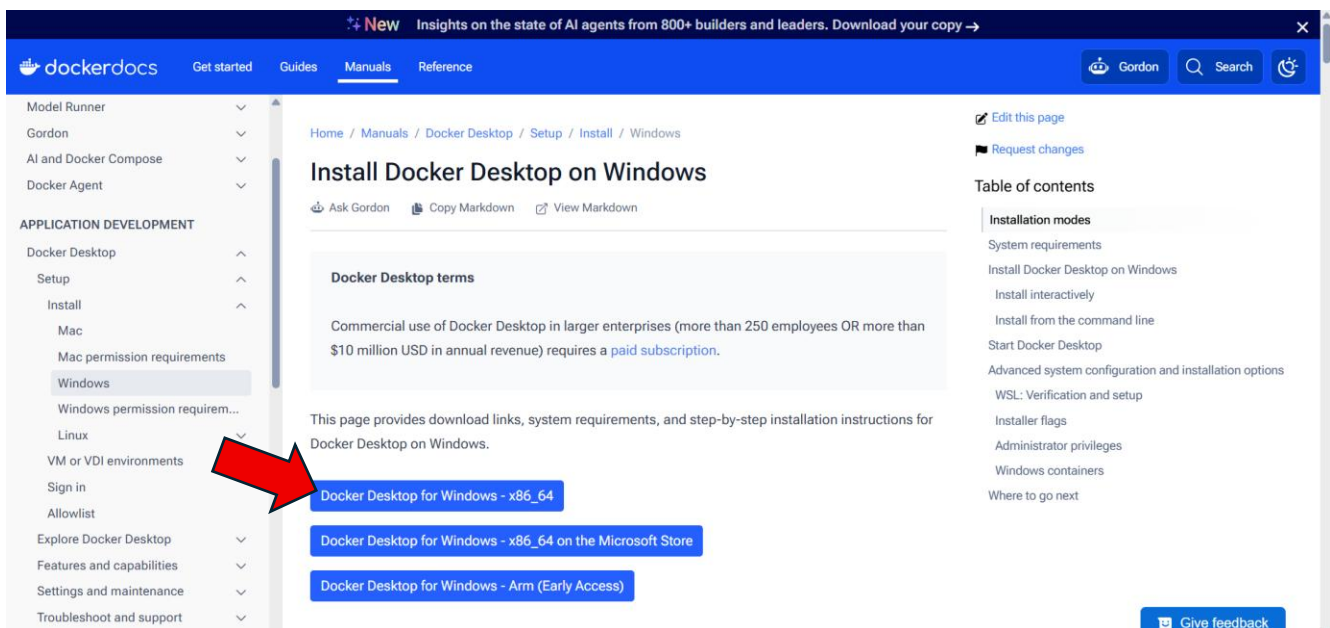
Vscode ติดตั้ง IDE ที่เป็น Editor เอาไว้เขียนโค้ดพัฒนาโปรแกรม

ขั้นตอน



Source : <https://code.visualstudio.com/>

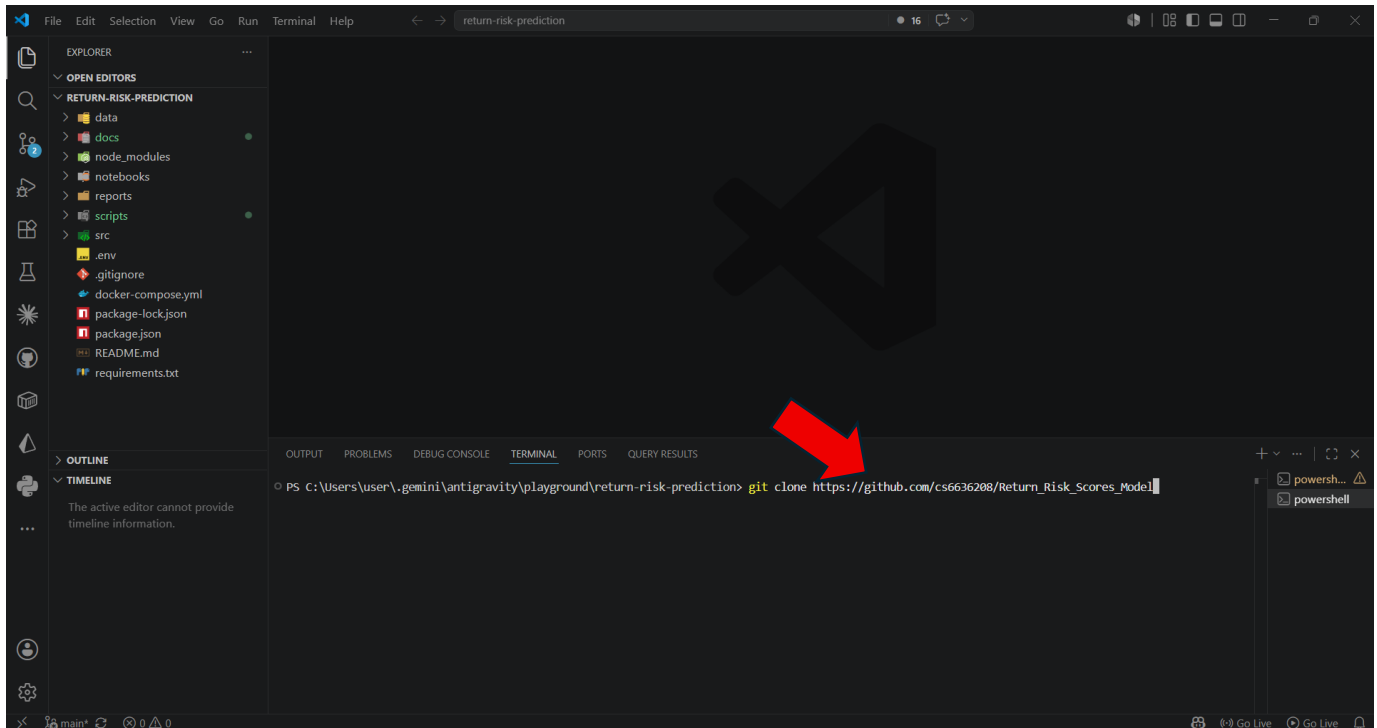
Docker Desktop เราจะรันฐานข้อมูลหรือ Database โดยรันผ่านตัว Docker แล้วขึ้น PostgreSQL



เปิดเข้า Vscode แล้วกดที่ tab bar ด้านบนที่เขียนว่า terminal -> new terminal เปิดขึ้นมาเสร็จแล้วก็ Clone Project มาโดยใช้คำสั่งนี้ที่ terminal ตามรูป

ถ้าต้องการนำโปรเจกต์ทั้งหมดที่ทำมาไว้ในเครื่องเราใช้คำสั่งที่บน terminal :

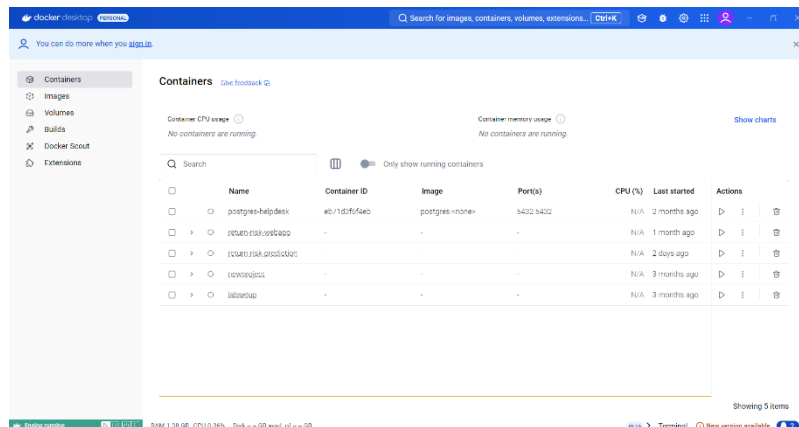
git clone https://github.com/cs6636208/Return_Risk_Scores_Model



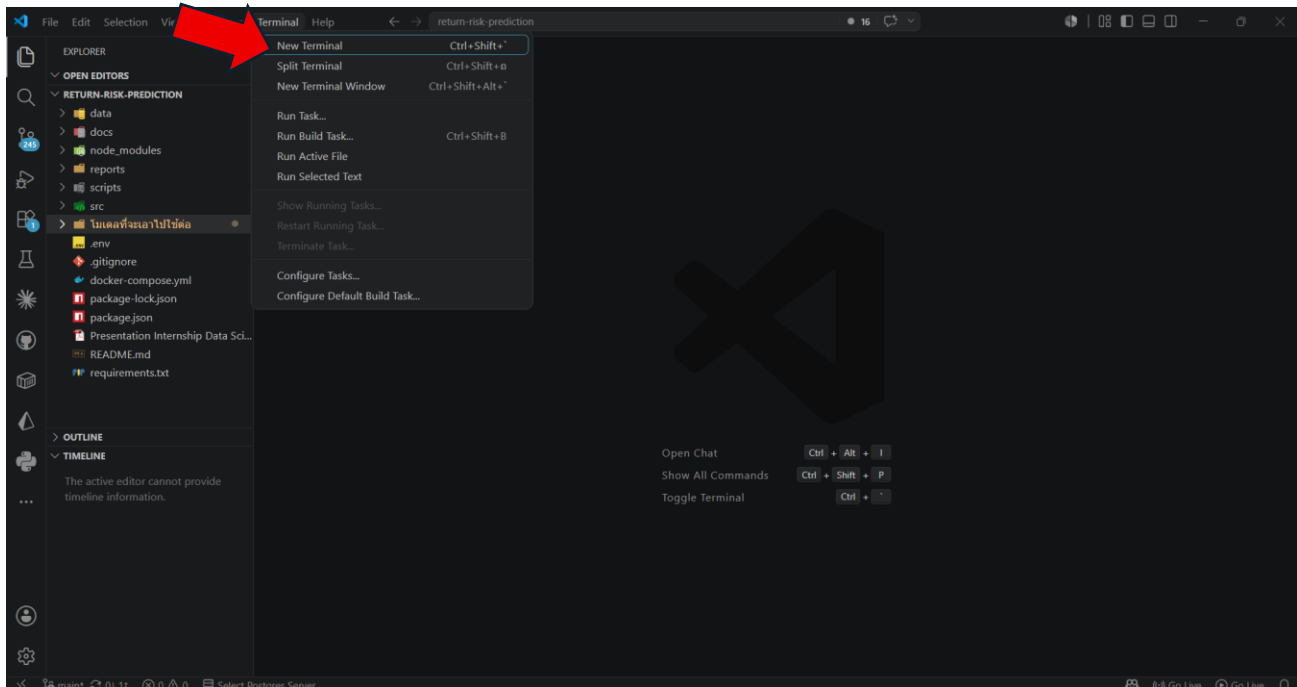
ติดตั้ง library python โดยใช้คำสั่ง pip install -r requirements.txt เพื่อ set เอาไว้จะได้โหลด library ที่ python จำเป็นต้องใช้งานได้

วิธีเปิดฐานข้อมูล PostgreSQL โดยใช้ Docker Desktop

Step 1 : เปิดตัว Docker Desktop login ลงชื่อเข้าใช้ให้เรียบร้อยจากนั้นรอโหลดเสร็จขึ้นหน้าหลัก ตามรูป

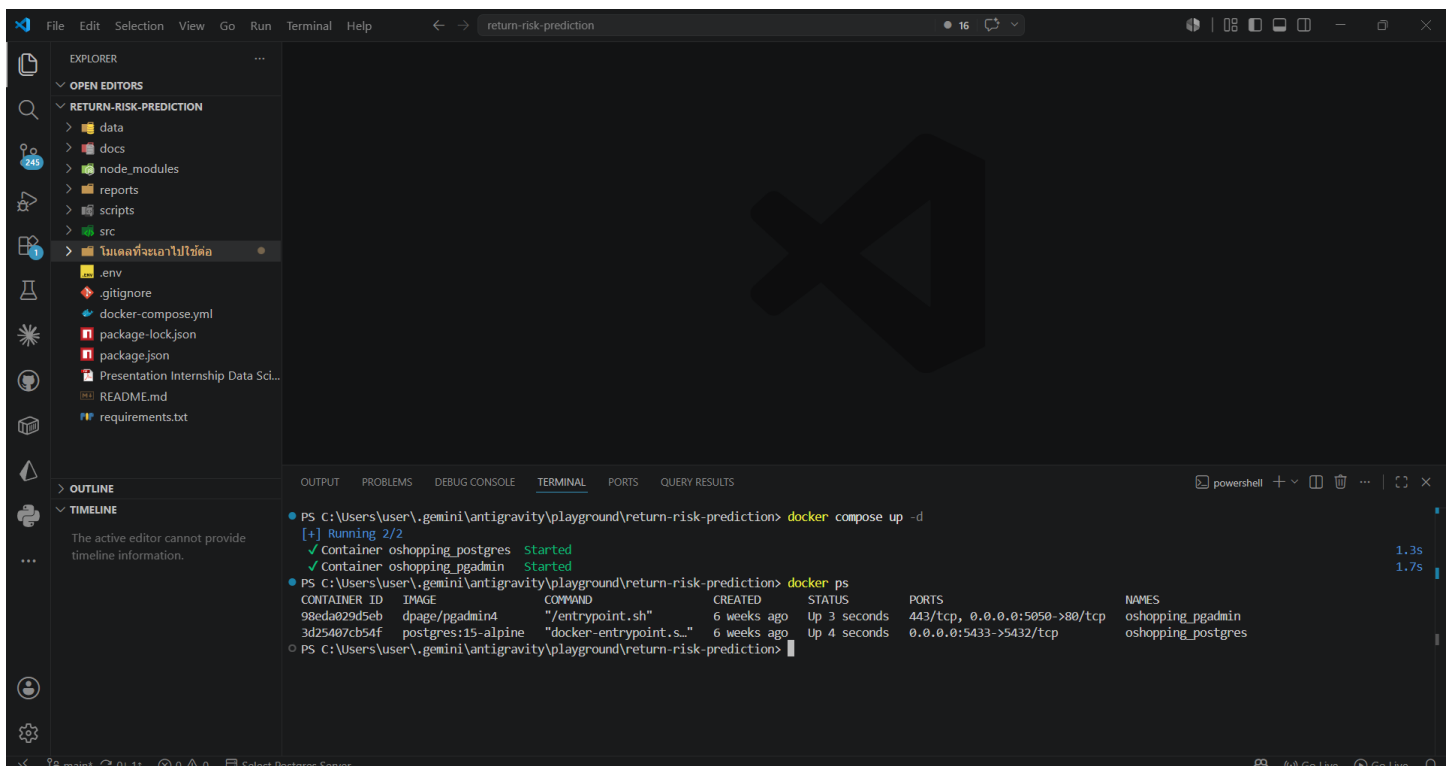


Step 2 : ไปที่ Vscode เปิด Command Prompt ที่ terminal ของ Vscode ขึ้นมา จากนั้นพิมพ์คำสั่งตามบรรทัด



docker compose up -d เพื่อเปิด server ฐานข้อมูล PostgreSQL (ถ้ามี Database อยู่แล้วสามารถใช้ Database โหลดขึ้น PostgreSQL ได้เลย)

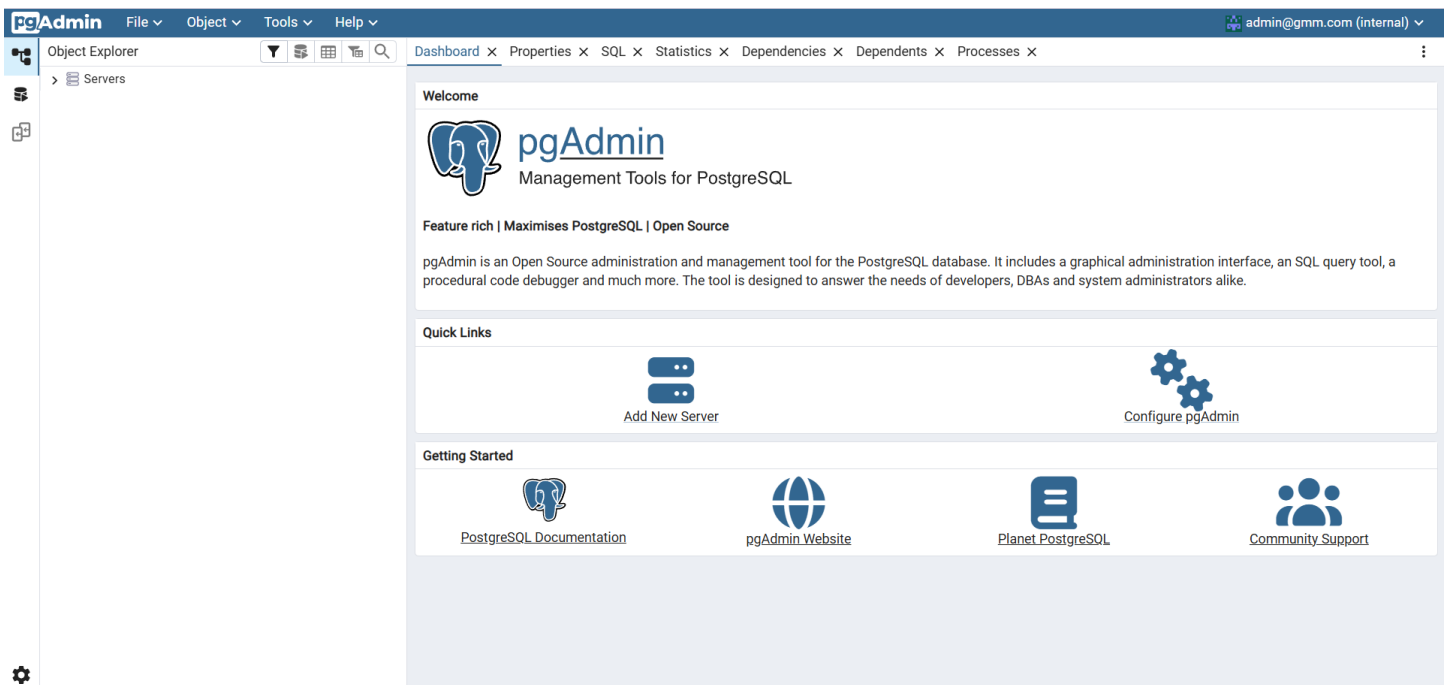
docker ps เช็คสถานะ Postgre ขึ้นมารึยังถ้าขึ้นมาแล้วให้พิมพ์ที่ google ว่า : <http://localhost:5050>



จะได้หน้าต่างขึ้นมาแบบนี้ ตามรูป



จากนั้นใส่ username : admin@gmm.com และ password : admin (กรณีมี Database ของบริษัทอยู่แล้วให้ใช้ของบริษัท)



สรุปโปรเจกต์แบบสั้น

- 1. โมเดลที่เลือกใช้คือ LightGBM Version 5 หรือ V5 ซึ่งเป็นโมเดลสำหรับทำนายว่า order ใหม่มีโอกาสคืนสินค้าสูงหรือต่ำ โมเดลนี้ไม่ได้ตัดสินแทนคนทั้งหมด แต่ใช้เป็นเครื่องมือช่วยเตือนให้ทีมงานเห็น order ที่ควรตรวจละเอียดขึ้น เช่น โทรยืนยันก่อนส่ง หรือเช็คข้อมูลสินค้าและลูกค้าเพิ่มเติม
- 2. ชุดข้อมูลที่ใช้ train หลัก: SETC/S1 clean_dataset_s1.csv จำนวน 5,000 rows
- 3. ชุดข้อมูลที่ใช้ test ภายนอก: SETD/S3 real_dataset_s1.csv จำนวน 55,000 rows
- 4. โมเดลที่เลือก: LightGBM V5 เพราะผลทำนายค่อนข้างนิ่ง ใช้ feature ไม่เยอะเกินไป และเหมาะสมเป็น baseline สำหรับทำระบบจริงต่อ
- 5. สถานะโปรเจกต์ตอนนี้: ส่วนโมเดลทดลองและเอกสารประกอบพร้อมใช้ต่อแล้ว แต่ถ้าจะใช้กับข้อมูลบริษัทจริง ต้อง map feature และ retrain อีกครั้ง

เนื่องจากโปรเจกต์ที่ทำอยู่ตอนนี้เน้นไม่ใช่ข้อมูลจริงของบริษัทเป็นข้อมูลที่ถูกสร้างมาจาก Gen data ทั้ง clean แลพ real dataset ทำให้เมื่อได้ข้อมูลจริงจากทางบริษัทเข้ามาแล้วต้องทำการตรวจเช็คดูว่า feature ที่ระบบนี้ ทำกับ feature ที่ข้อมูลจริงเข้ามตรงกันหรือไม่ ถ้าไม่ก็ต้อง retrain แล้วนำของจริงมา test กับ model ที่ train เสร็จแล้ว

Feature ที่โมเดล V5 ใช้คืออะไร

กลุ่ม feature	ตัวอย่าง	ความหมายแบบง่าย
ข้อมูลลูกค้า	age, membership_tier, province	ลูกค้าอยู่กลุ่มไหน อายุเท่าไร อยู่จังหวัดใด
ข้อมูลสินค้า/order	category, brand, quantity, total_amount	ซื้อสินค้าอะไร ราคาเท่าไร จำนวนเท่าไร

ประวัติลูกค้า	total_orders_before, total_returns_before, customer_return_ratio	ลูกค้าคนนี้เคยซื้อและเคยคืนบ่อยแค่ไหน
ประวัติย้อนหลังตามช่วงเวลา	hist_return_rate_7d, 30d, 90d, 365d	ช่วงหลังๆ ลูกค้าคืนบ่อยขึ้นหรือลดลง
ความเสี่ยงสินค้า/หมวด	product_return_rate_pti, category_return_rate_pti, brand_return_rate_pti	สินค้าหรือหมวดนี้เคยมีการคืนสูงไหม
ความเสี่ยงขนส่ง/ช่องทาง	courier_return_rate_pti, payment_return_rate_pti, channel_return_rate_pti	ขนส่ง วิธีจ่ายเงิน หรือช่องทางขายนี้มี pattern คืนสินค้าไหม
feature ผสม	category_payment, high_discount_cod, low_rating_high_discount	จับคู่ปัจจัย เช่น หมวดสินค้า + COD หรือส่วนลดสูง + rating ต่ำ

ถ้าโมเดลทำนายพลาด ต้องแก้ตรงไหน

การทำนายพลาดเกิดได้ปกติ เพราะโมเดลเรียนรู้จาก pattern ในอดีต ไม่ได้รู้ความจริงล่วงหน้า

วิธีแก้ต้องดูว่าพลาดแบบไหน

ปัญหา	ความหมาย	ควรตรวจ/แก้ตรงไหน
โมเดลบอกเสี่ยงสูง แต่ลูกค้าไม่คืน	False Positive	threshold อาจต่ำเกินไป, feature บางตัวให้ความเสี่ยงสูงเกินจริง, กลุ่มลูกค้า/สินค้าอาจเปลี่ยน pattern
โมเดลบอกเสี่ยงต่ำ แต่ลูกค้าคืนจริง	False Negative	โมเดลจับเคสเสี่ยงไม่ทัน, อาจต้องเพิ่ม feature เช่น return reason, product

		defect, call center note หรือปรับ threshold ให้เข้มข้น
ข้อมูลใหม่มี category/courier/payment ที่ไม่เคยเจอ	ข้อมูลใหม่ไม่เหมือนตอน train	ต้อง update mapping, เพิ่มค่าใหม่ใน feature builder และ retrain
Accuracy ตกเมื่อใช้ข้อมูลจริง	ข้อมูลจริงต่างจากข้อมูลทดลอง	ทำ EDA เทียบ distribution, ตรวจสอบ missing/outlier, retrain ด้วยข้อมูลจริง
ระบบทำนายช้า	feature engineering ใช้เวลานาน	ใช้ Feature Store/cache เก็บค่าประวัติลูกค้าและ return rate ไว้ล่วงหน้า

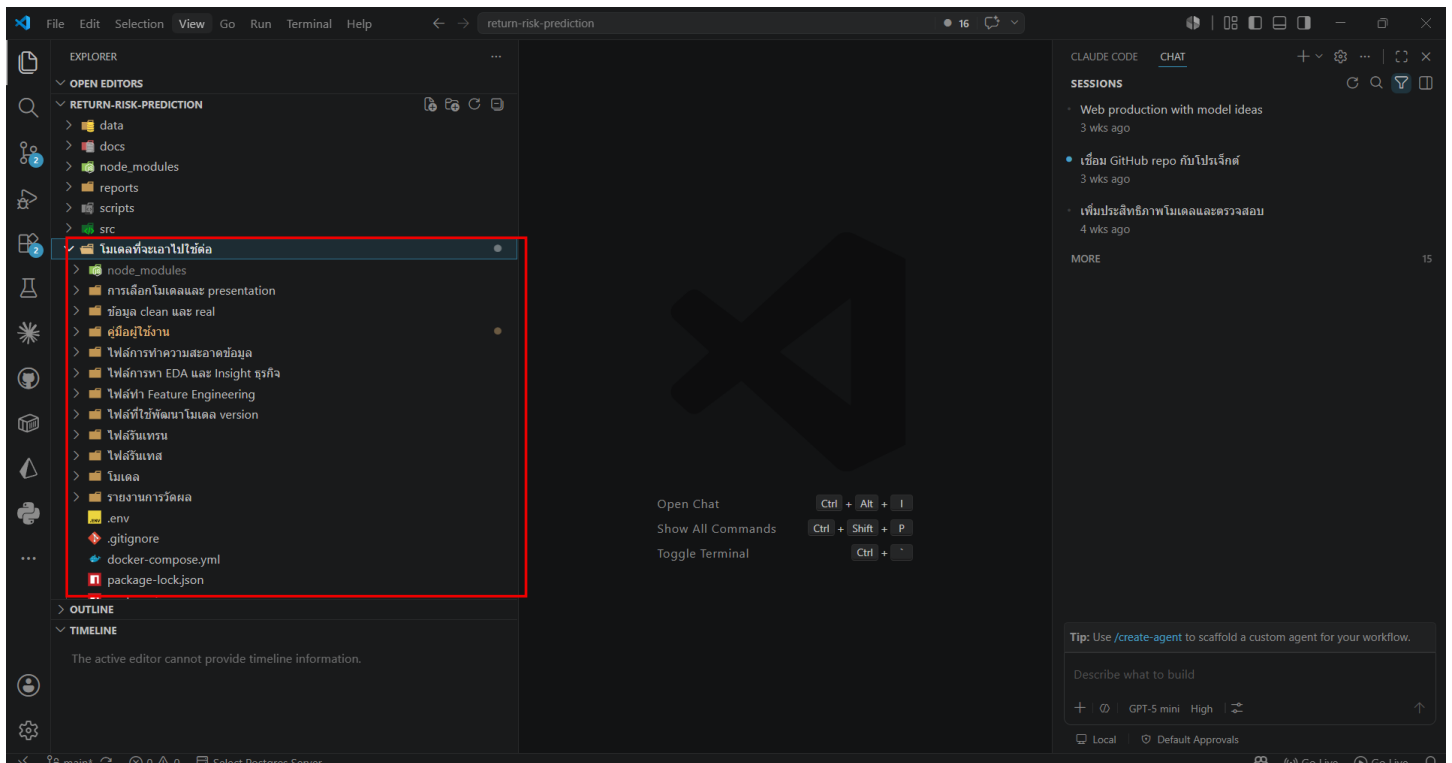
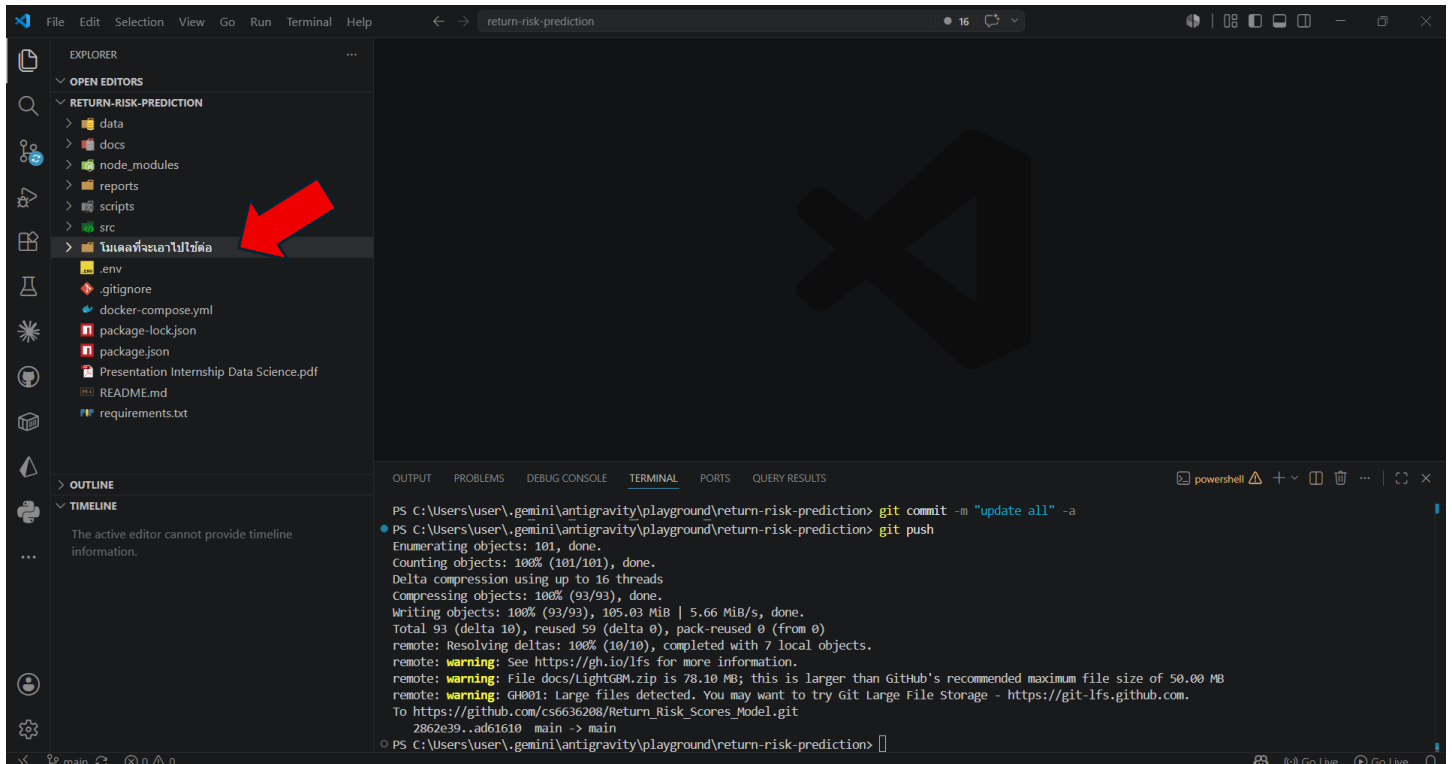
ทำไมถึงเลือก LightGBM V5

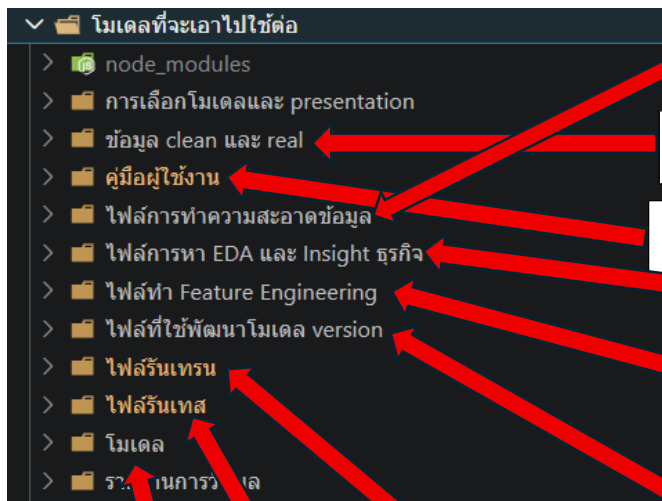
V5 เป็นเวอร์ชันที่เลือก feature แบบพอดี ไม่เบาเกินไปแบบ V1 และไม่หนักเกินไปแบบ V4 จุดเด่นคือยังเก็บ feature สำคัญ เช่น ประวัติลูกค้า ความเสี่ยงสินค้า ความเสี่ยงหมวดสินค้า วิธีชำระเงิน และ interaction บางตัวไว้ แต่ตัด feature ที่ทำให้ระบบหนักหรือช้าซ้อนออก

ชุดทดสอบ	จำนวนข้อมูล	Accuracy	Recall	F1	AUC	หมายเหตุ
SETC S1 Holdout	1,000 rows จาก clean 5,000	82.00%	59.56%	67.86%	83.27%	ใช้วัดหลัง train โดยแยก holdout
SETD S3 External	55,000 rows	81.91%	61.24%	68.37%	82.59%	ใช้ดูว่าโมเดลเจอข้อมูลใหม่แล้วยังนิ่งไหม

โครงสร้างโฟลเดอร์และไฟล์

เมื่อเรา Clone โปรเจกต์ทั้งหมดมาได้แล้ว โครงสร้างโฟลเดอร์และไฟล์ ตามรูป





Folder รวบรวมกระบวนการทำ clean data เมื่อมี raw data
เอามาทำ clean

Folder เก็บข้อมูล clean และ real

Folder เก็บคู่มือการใช้งานในโปรเจกต์นี้

Folder รวบรวมไฟล์และรูปภาพที่หา Insight ที่
ได้มาทั้งหมดจาก return

Folder รวบรวมไฟล์และรูปภาพที่หา Insight ที่
ได้มาทั้งหมดจาก return

Folder ที่เก็บ clean กับ real dataset ที่ใช้ model lightgbm ใน
การพัฒนาแต่ละ version ตั้งแต่ version 1 - 5

Folder ที่เก็บข้อมูลไฟล์ python ที่ใช้ train model โดยใช้ข้อมูล clean
data และ feature ที่จะใช้เข้ามา train (ใช้ clean_dataset version 5 ที่
5000 มา train) clean_dataset_s1.csv (5000 rows)

Folder ที่เก็บข้อมูลไฟล์ python ที่ใช้ test model โดยใช้ข้อมูล
real_data และ feature ที่จะใช้เข้ามา test (ใช้ real_dataset version 5
มา test) real_dataset_s1.csv (55000 rows)

Folder ที่เก็บโมเดลที่จะเลือกใช้ในการ
ทำโปรเจกต์นี้ในที่นี้ใช้ Model
LightGbm version 5 ที่ clean_dataset
5000 กับ real_dataset 55000 (Model
นี้รวบรวมมาจาก best feature ทั้ง 1 -
4 version เกิดมาเป็น version 5)

คำอธิบายคำศัพท์

คำ	ความหมายแบบง่าย
Model	ตัวช่วยทำนายจากข้อมูลในอดีต
Feature	ข้อมูลที่แปลงแล้วเพื่อให้โมเดลใช้คิด
Train	ให้โมเดลเรียนรู้จากข้อมูลที่มีคำตอบแล้ว
Test	นำโมเดลไปสอบกับข้อมูลอีกชุดเพื่อดูว่าทายแม่นยำแค่ไหน
Accuracy	ทายถูกทั้งหมดกี่เปอร์เซ็นต์
Recall	จับเคสที่คืนสินค้าได้มากแค่ไหน
Threshold	เส้นแบ่งว่าจะนับว่าเสี่ยงหรือไม่เสี่ยง
Retrain	ฝึกโมเดลใหม่ด้วยข้อมูลชุดใหม่
Feature Store	ที่เก็บ feature ที่คำนวณไว้ล่วงหน้า เพื่อให้ระบบทำนายเร็วขึ้น

สำหรับไฟล์ `run_test_v5_lightgbm.py` ถ้าจะเอา Data จริงบริษัทมาใช้
ก่อนอื่นถ้าจะเอา Data จากบริษัทจริงมา test กับ model version 5 lightgbm ที่ `clean_dataset_s1.csv` 5000
ต้องไปทำ feature แล้วได้ผลลัพธ์ที่ใช้อย่างมาเข้า test model ในที่นี้โปรเจกต์ผมใช้ผล test จาก
`real_dataset_s1.csv`

1. ต้องทำ Feature Engineering ก่อน

`run_test_v5_lightgbm.py` ไม่ควรรับ raw data ตรงๆ เช่น `order_id`, `customer_id`, `product_id` แล้ว predict
ทันที เพราะโมเดล V5 ต้องการ feature 64 ตัวที่ผ่านการแปลงแล้ว

ให้เอาข้อมูลจริงไปรันไฟล์นี้ก่อน:

`[feature_engineering_v5_lightgbm.py]/C:/Users/user/.gemini/antigravity/playground/return-risk-prediction/โมเดลที่จะเอาไปใช้ต่อ/ไฟล์ทำ Feature Engineering/feature_engineering_v5_lightgbm.py)`

ตัวอย่าง:

```
python "โมเดลที่จะเอาไปใช้ต่อ/ไฟล์ทำ Feature Engineering/feature_engineering_v5_lightgbm.py" --  
input-csv "path\to\company_real_data.csv"
```

ผลลัพธ์จะได้ไฟล์ feature ที่พร้อมเข้าโมเดล เช่น df_featured_v5_from_input.csv

2. ตอน run_test ให้เปลี่ยน input-csv

จากเดิม script ใช้ไฟล์ feature ตัวอย่างเดิม ให้ชี้ไปที่ไฟล์ feature ของบริษัทจริง:

```
python "โมเดลที่จะเอาไปใช้ต่อ/ไฟล์รันเทส/run_test_v5_lightgbm.py" --mode model --input-csv "โมเดลที่  
จะเอาไปใช้ต่อ/ไฟล์ทำ Feature Engineering\outputs\df_featured_v5_from_input.csv"
```

จุดในโค้ดที่เกี่ยวข้องคือ:

```
[run_test_v5_lightgbm.py (line 244)](C:/Users/user/.gemini/antigravity/playground/return-risk-  
prediction/โมเดลที่จะเอาไปใช้ต่อ/ไฟล์รันเทส/run_test_v5_lightgbm.py:244)
```

```
parser.add_argument("--input-csv", default=str(DEFAULT_FEATURED_DATA_PATH))
```

ไม่จำเป็นต้องแก้โค้ดถ้าใช้ --input-csv ตอนรัน แต่ถ้าอยากตั้งค่า default ถาวร ให้เปลี่ยน

```
DEFAULT_FEATURED_DATA_PATH
```

3. ถ้าข้อมูลจริงมีคำตอบ is_returned แล้ว จะวัด Accuracy ได้

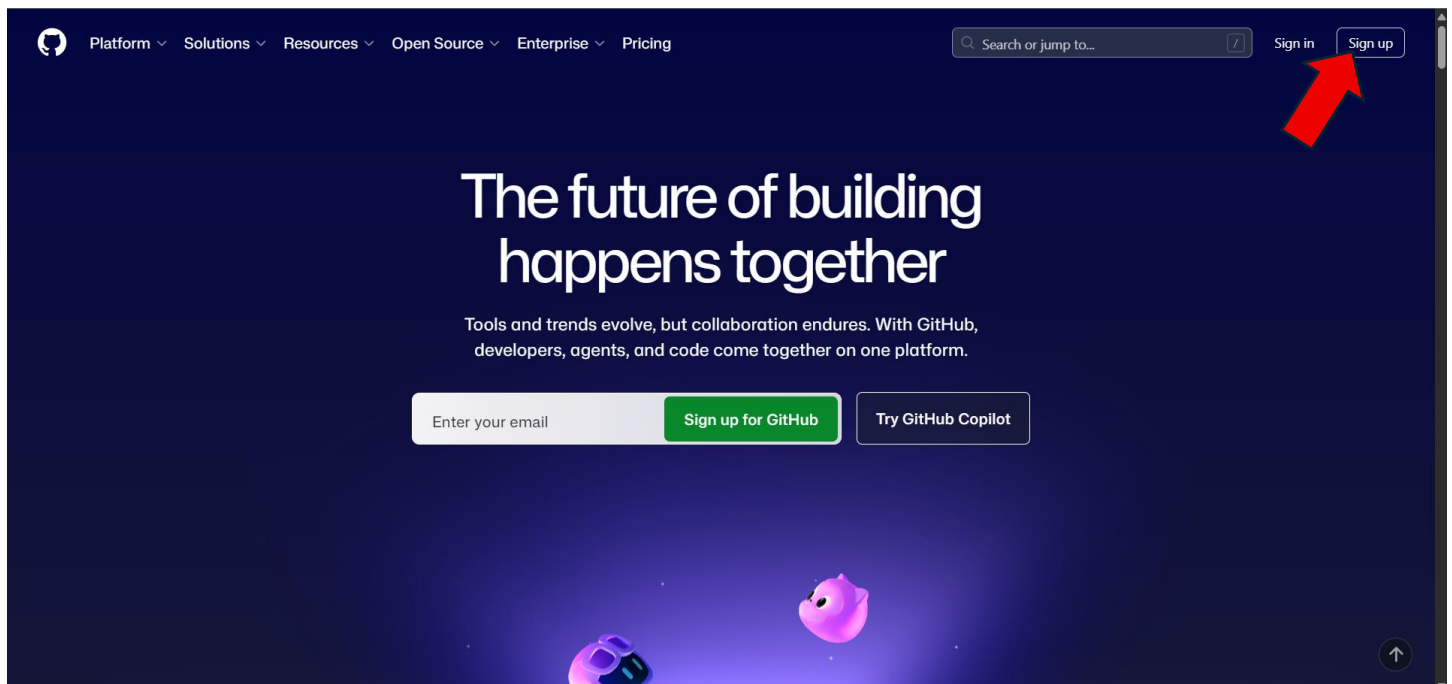
ถ้าไฟล์ feature ของบริษัทจริงมี column: is_returned

script จะคำนวณ Accuracy, Recall, Precision, F1, Cost ให้ได้

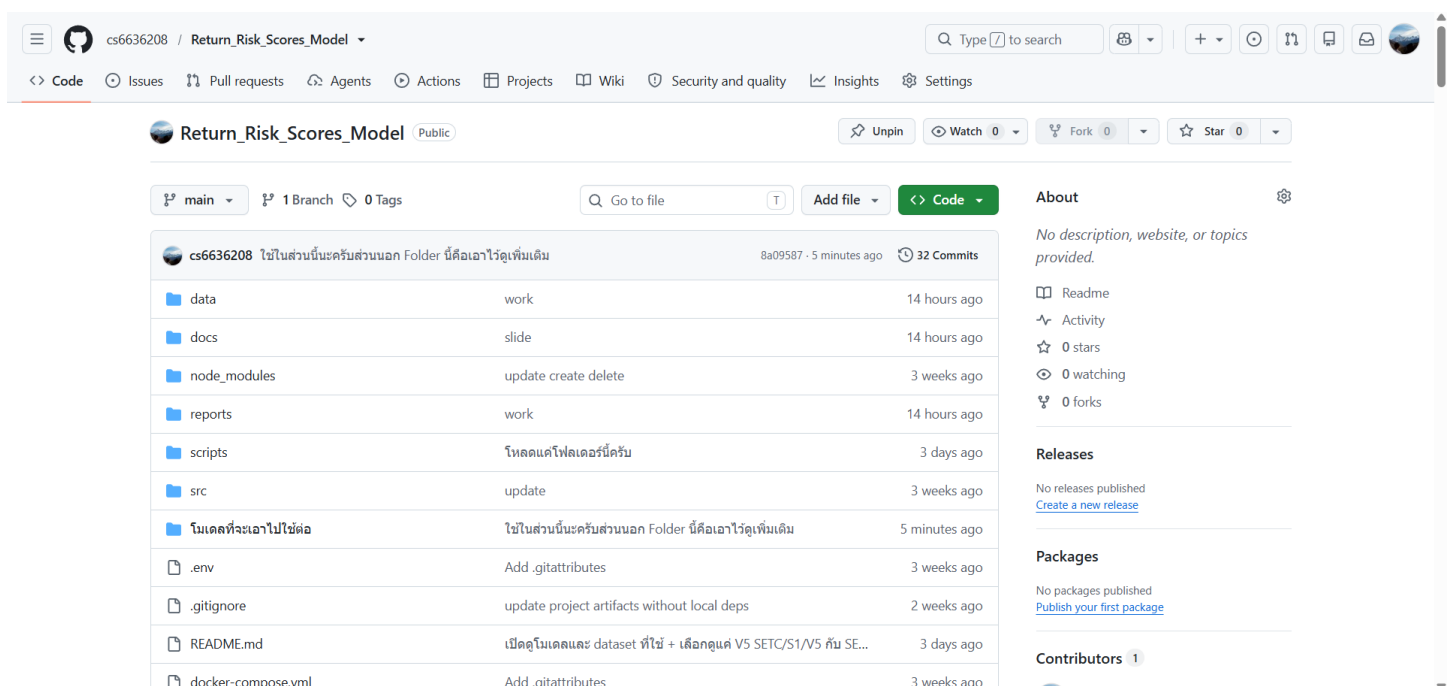
แต่ถ้าเป็น order ใหม่ที่ยังไม่รู้ว่าขึ้นไหม จะไม่มี is_returned แบบนี้ script จะทำได้แค่ predict ว่าเสี่ยงขึ้นกี่ %
ยังวัด Accuracy ไม่ได้

เพิ่มเติม

การใช้งาน Github ถ้าสมัครลงทะเบียนก่อนได้เลยครับ กด sign up เพื่อลงทะเบียนหรือถ้าไม่อยากลงก็ไม่เป็นไรครับกดเข้าดูที่หน้าเว็บ Github ผมได้เลยเพราะผมเปิดเป็น public ไว้ใครก็เข้าถึงได้ครับ



https://github.com/cs6636208/Return_Risk_Scores_Model Copy แล้วไปวางไว้ที่ Url Google ครับ



แล้วก็เลือกดู Folder ใต้ครับดูตรงแค่ โมเดลที่จะเอาไปใช้ต่อ

The screenshot shows a GitHub repository named 'Return_Risk_Scores_Model' by user 'cs6636208'. The left sidebar displays the file tree with the 'โมเดลที่จะเอาไปใช้ต่อ' folder selected. The main area shows the commit history for this folder, with a table listing recent commits.

Name	Last commit message	Last commit date
..		
การเลือกโมเดลและ presentation	work	14 hours ago
ข้อมูล clean และ real	update all	18 hours ago
คู่มือผู้ใช้งาน	ใช้ในส่วนนี้จะครบส่วนนอก Folder นี้คือเอาไว้ดูเพิ่มเติม	6 minutes ago
รายงานการวัดผล	update all	18 hours ago
โมเดล	โหลดแค่ไฟล์เดอร์นี้ครับ	3 days ago
ไฟล์การทำความสะอาดข้อมูล	work	14 hours ago
ไฟล์การหา EDA และ Insight ธุรกิจ	work	14 hours ago
ไฟล์ทำ Feature Engineering	work	14 hours ago
ไฟล์ที่ใช้พัฒนาโมเดล version/LightGBM	work	14 hours ago
ไฟล์สไลด์	ใช้ส่วนนี้ครับ	21 minutes ago

ถ้าพี่จะทำโปรเจกต์ผมต่อให้เริ่มจาก run_test_v5_lightgbm.py ที่เราติดกันไว้ซึ่งผมทำไว้ให้แล้ว

ผมให้เอาไว้หมดแล้วถ้ามีติดขัดอะไรหรือไม่เข้าใจตรงไหนทักมาบอกผมเพิ่มเติมนะครับผมจะได้ช่วยพี่ได้

ช่วงนี้ผมอาจจะไม่ได้มีเวลาว่างเยอะนะครับเพราะผมต้องไปหาเลี้ยงทำเรื่องสอบหัวข้อแล้วก็จิตอาสาทุกสัปดาห์